

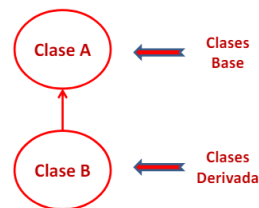
Poo - Herencia

☰ Categoría	segundaParte
⚙ Estado	Sin empezar
☰ Evaluaciones	



Permite crear una clase a partir de otra.

La figura siguiente indica que la clase B está heredando de la clase A.



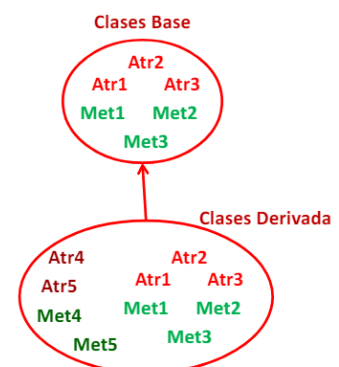
Reutilizar, heredar información de ciertas clases. Evita duplicidad de código, representar conocimiento de una manera más óptima.

1. Clase base
2. Clase derivada : se crea a partir de la clase base

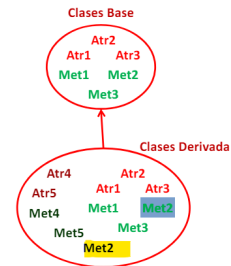
Ej: cuando nosotros nacemos heredamos características de nuestros padres.

▼ Características

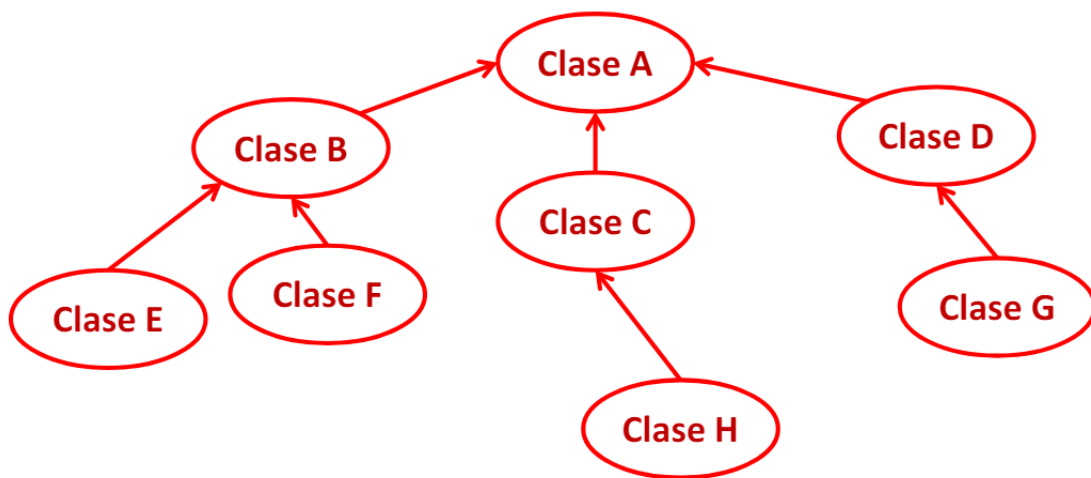
- La clase derivada adquiere todos los atributos y métodos que se deseen.
- Es posible ocultar o sobrescribir algún método. (no se destruye el método original, solo se oculta).
- Se tiene una versión del Met2 exclusivo para la clase derivada. **Por defecto se ejecuta el mét nuevo.**



- La clase derivada elige que información va a utilizar
- La clase base decide que información quiere **proteger** /mostrar

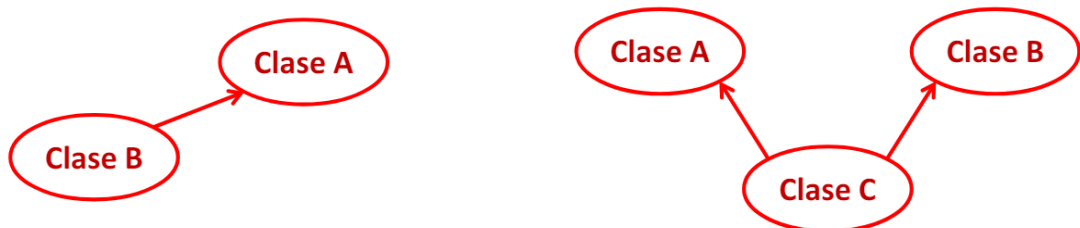


2. Desde la clase derivada no se puede acceder a los elementos privados de la clase base (atributos). Solo se tendrá acceso por los métodos.
3. Una clase puede heredar de otra clase. Otra clase puede heredar de la derivada → ARBOL O JERARQUIA DE CLASES



▼ Tipos de herencias

- HERENCIA SIMPLE: Cuando una clase hereda de una única clase
- HERENCIA MÚLTIPLE: Cuando una clase hereda de dos o más clases



▼ Zona protegida en la herencia

- En el curso siempre se trabajará con herencia **public**.



Solo tiene efecto en herencia. Se pueden colocar, tanto atributos, como métodos.

- El mismo objeto no puede acceder a la zona protegida, para ellos es una zona privada.
- Una clase derivada, sí puede acceder a la zona protegida de su clase base. (desde pokebola sí se podría acceder a través de un método).
- Se utiliza como una puerta falsa para acceder directamente a los atributos de la clase base

Implementación:

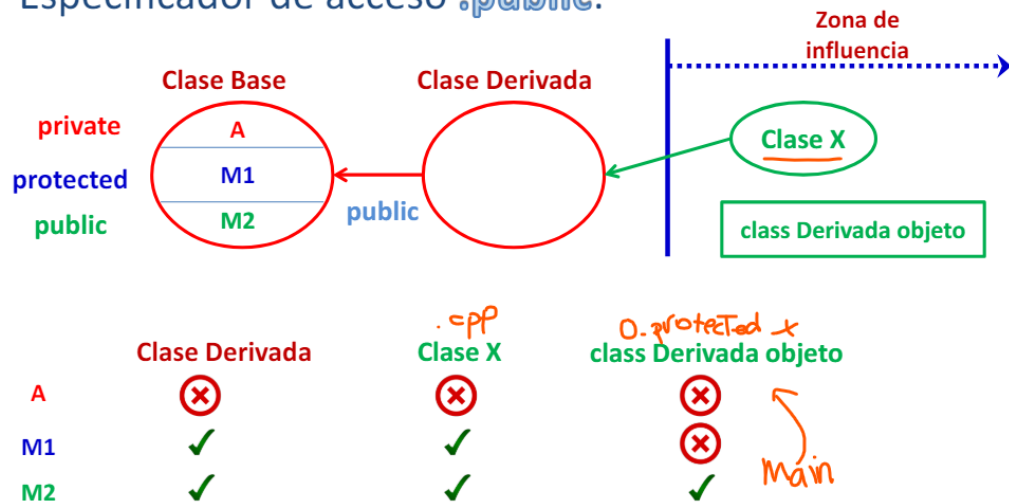
```
class Base{
    private:
        ...
    public:
        ...
    protected:
        ...
}

class Derivada:
    public Base{
        private:
            ...
        public:
            ...
            Metodo()
    }
```

▼ Especificadores de acceso en la herencia

- **PUBLIC:**
 - `Pokeball` hereda todos los miembros públicos y protegidos de `Item`.
 - Los miembros privados de `Item` no son accesibles directamente desde `Pokeball`, pero `Pokeball` puede acceder a ellos usando los métodos públicos o protegidos de `Item` que accedan a estos miembros privados.

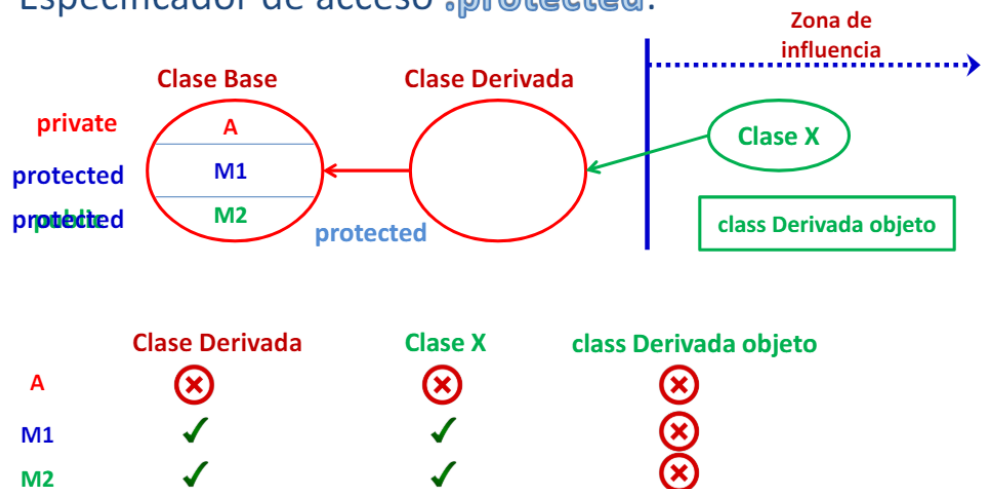
Especificador de acceso :public.



[26]

- **PROTECTED:** Cuando declaramos la herencia como protected , no se podrá acceder a ninguna zona en el main. Y en el .cpp solo a los metodos-atributos protected.
 1. Los miembros protegidos de `Item` (`protectedVar`) son accesibles directamente desde `Pokeball` como miembros protegidos.
 2. Los miembros públicos de `Item` (`publicVar` y `getPrivateVar()`) se convierten en miembros protegidos de `Pokeball` . Esto significa que estos miembros públicos de `Item` no serán accesibles desde fuera de `Pokeball` , pero sí serán accesibles dentro de `Pokeball` y de cualquier clase que herede de `Pokeball` .
 3. Los miembros privados de `Item` (`privateVar`) no son accesibles directamente desde `Pokeball` , pero `Pokeball` puede acceder a ellos mediante métodos públicos o protegidos de `Item` .

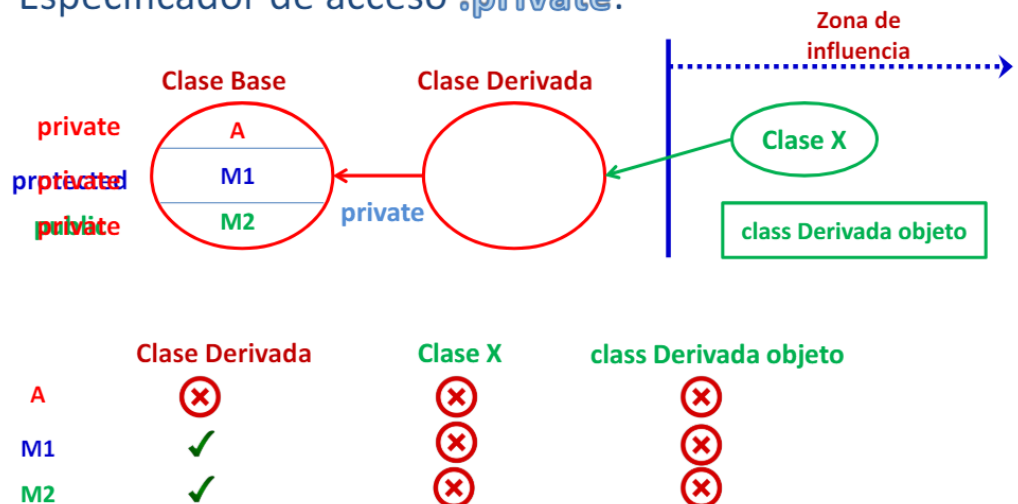
Especificador de acceso :protected.



• **PRIVATE:** sfdfsadaffdf

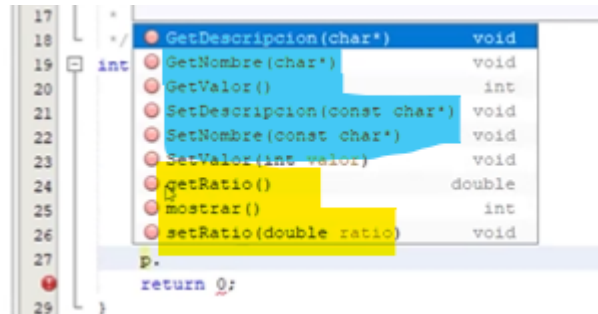
1. Los miembros protegidos de `Item` (`protectedVar`) se convierten en miembros privados de `Pokeball`. Esto significa que estos miembros no serán accesibles desde fuera de `Pokeball`, ni siquiera por clases que hereden de `Pokeball`.
2. Los miembros públicos de `Item` (`publicVar` y `getPrivateVar()`) también se convierten en miembros privados de `Pokeball`. Estos miembros no serán accesibles desde fuera de `Pokeball`, ni siquiera por clases que hereden de `Pokeball`.
3. Los miembros privados de `Item` (`privateVar`) no son accesibles directamente desde `Pokeball`, pero `Pokeball` puede acceder a ellos mediante métodos públicos o protegidos de `Item` que ahora son privados en `Pokeball`.

Especificador de acceso :private.



▼ Clase Erasmo

La clase derivada tiene acceso a sus propios métodos como a los métodos de la clase padre



Primero se coloca a la clase hija, luego a la clase padre

```
#include "Pokeball.h" // CLASE DERIVADA - HIJA
#include "Item.h"      // CLASE PADRE
```

▼ Ejemplo del cilindro

- Herencia a segundo nivel
- Ahorra código